KITCHEN VISION

PROJECT DESCRIPTION

Kitchen Vision – AI-Powered Pantry Analysis and Smart Recipe Recommendation System is an intelligent cooking assistance application developed using Python and the Streamlit framework. The project focuses on transforming a simple pantry image into useful cooking insights by applying computer vision and artificial intelligence techniques.

The system allows users to upload an image of their pantry. The uploaded image is pre-processed using enhancement techniques such as resizing and contrast adjustment to improve detection accuracy. A vision-based AI module then analyzes the image and identifies visible food ingredients present in the pantry.

After detecting the ingredients, the system categorizes them into storage types such as Dry Storage, Cold Storage, Frozen, and Fresh Produce. This classification helps users understand the organization of their available ingredients and improves ingredient management.

Using the detected ingredients, the system generates three simple recipes suitable for beginners. Each recipe contains a list of ingredients, three-step cooking instructions, preparation time, and difficulty level. If any ingredients required for a recipe are missing, the system automatically generates a grocery list.

The application also provides substitution suggestions using a predefined substitution knowledge base, allowing users to replace unavailable ingredients with suitable alternatives.

To promote healthier meal choices, Kitchen Vision includes a health evaluation module that assigns a nutritional score from 1 to 10 for each recipe. The score is calculated based on factors such as ingredient composition, calorie estimation, protein content, fiber availability, and fat levels. A short explanation is also provided with the score.

The system follows a modular architecture where image processing, vision detection, recipe generation, substitution logic, and health evaluation are organized into separate modules. This modular design improves scalability and makes the application easier to maintain and extend.

Overall, Kitchen Vision demonstrates how artificial intelligence can simplify meal planning, reduce food waste, and encourage healthier cooking habits by transforming pantry images into meaningful cooking recommendations.

USE CASE SCENARIOS

Scenario 1: Pantry Image Analysis and Item Detection

Users open the Kitchen Vision application and upload an image of their pantry. The system preprocesses the image using enhancement and resizing techniques to improve clarity and detection accuracy. The AI-powered vision module then analyzes the image and detects visible food ingredients. After processing, a structured list of identified ingredients is generated and displayed to the user through the application interface.

Scenario 2: Item Categorization and Storage Classification

Once the ingredients are detected, the system automatically classifies each item into appropriate storage categories such as Dry Storage, Cold Storage, Frozen, or Fresh Produce. This categorized inventory is displayed in organized sections, helping users clearly understand the available ingredients and their storage types.

Scenario 3: Smart Recipe Generation with Health Scoring

Based on the categorized pantry items, the system generates three simple and beginner-friendly recipes. Each recipe includes a list of ingredients, three-step cooking instructions, estimated preparation time, and difficulty level. At the same time, the Health Evaluation Service analyzes the recipe composition and assigns a nutritional score along with a brief explanation.

Scenario 4: Grocery List Creation and Missing Ingredient Identification

The system compares the ingredients required for each generated recipe with the items currently available in the pantry. Any missing ingredients are automatically identified and compiled into a grocery list. The list is displayed in a checklist format, making it easier for users to track the items they need to purchase.

Scenario 5: Ingredient Substitution Recommendation

For ingredients that are missing, the system suggests suitable substitutes using a predefined substitution knowledge base. These alternatives allow users to continue preparing recipes even if some ingredients are unavailable.

Scenario 6: Consolidated Cooking Guidance Output

Finally, the application presents all results in a clean and structured interface. Detected ingredients, categorized storage groups, generated recipes, health scores, grocery lists, and substitution suggestions are displayed clearly, enabling users to make quick and informed cooking decisions.

ARCHITECTURE OVERVIEW

Kitchen Vision is an intelligent application designed to analyze pantry images and generate smart cooking recommendations. The system combines a **Streamlit-based frontend** with an **AI-powered backend** that processes images and produces structured cooking insights. The architecture follows a **modular service-oriented design**, where image processing, item detection, recipe generation, substitution suggestions, and health evaluation are handled by separate modules. This design improves system scalability, maintainability, and performance.

The platform supports both **mock AI detection and real multimodal AI models**, such as Gemini Vision or OpenAI Vision. This flexibility allows developers to easily switch between models depending on system requirements and availability.

Core Technologies

- **Streamlit:** Builds the interactive user interface.
- **Python:** Main programming language for system logic.
- **Pillow (PIL):** Performs image preprocessing and enhancement.
- **Gemini Vision / OpenAI Vision (Optional):** Detects pantry items using multimodal AI.
- **Heuristic AI Logic:** Handles recipe generation, substitution reasoning, and health scoring.
- **Python-dotenv:** Manages environment variables securely.
- **Docker:** Supports containerized deployment.

System Components

Component	Description
Frontend UI	Allows users to upload pantry images and view results.
Image Processing Module	Enhances and resizes images for better detection.
Vision Service Layer	Detects pantry ingredients using AI or mock detection.
Item Categorization Module	Classifies items into Dry, Cold, Frozen, or Fresh categories.
Recipe Generation Service	Generates simple recipes with instructions.

Component	Description
Substitution Service	Suggests alternatives for missing ingredients.
Health Evaluation Service	Assigns nutritional health scores for recipes.
Configuration Layer	Manages system settings and API keys.

System Output

The application displays results directly in the interface, including:

- Detected pantry ingredients
- Storage-based categorization
- Three-step recipe suggestions
- Health scores with explanations
- Grocery list of missing items
- Suggested ingredient substitutions

Pre-requisites

- Python 3.8+
- Streamlit Framework
- AI Provider Account (Optional) – Gemini API or OpenAI Vision
- Docker (Optional) for container deployment
- Required Python libraries listed in requirements.txt

Project Workflow

1. AI Vision Setup
    - Create Google AI Studio / OpenAI account
    - Generate API key for Vision API
    - Check model availability and limits
2. Configure Environment
    - Create .env file with API key
    - Set model name and parameters
    - Configure image size and compression
3. Prompt Design
    - Prompts for pantry item detection
    - Prompts for recipe generation
    - Prompts for ingredient substitutions
    - Prompts for health score explanation
    - Test with sample pantry images
4. Core Development
    - Create modular project structure (UI, services, utils)
    - Implement image processing (resize, compress, convert)
    - Build Vision Service for ingredient detection
    - Add mock detection mode
    - Return structured ingredient list

Activity 2.4: Item Categorization

- Create storage categories
- Assign items to Dry, Cold, Frozen, Fresh

Activity 2.5: Recipe Generation

- Generate 3 simple recipes
- Add ingredients, steps, time, difficulty
- Show missing ingredients

Activity 2.6: Substitution Service

- Find substitutes for missing items
- Show replacement suggestions

Activity 2.7: Health Evaluation

- Give health score (1–10)
- Provide short explanation

---

3. Backend Implementation (Streamlit)

Activity 3.1: Streamlit Interface

- Upload image
- Show image preview
- Add buttons to run process

Activity 3.2: Connect Services

- Connect Vision service
- Connect Recipe service
- Connect Substitution service
- Connect Health service

Activity 3.3: Data Validation

- Check AI results
- Handle empty results
- Format data properly

Activity 3.4: Error Handling

- Use try–except
- Show simple error messages
- Keep debug logs

4. UI Development

Activity 4.1: Streamlit Layout

- Add title and description
- Create image upload section

- Add results section

Activity 4.2: Show Results

- Display detected items list
- Show storage categories
- Show recipe cards
- Show health score
- Show grocery checklist

Activity 4.3: Improve User Experience

- Add loading spinner
- Use expandable sections
- Add clear labels

---

5. Testing & Optimization

Activity 5.1: Functional Testing

- Test with different pantry images
- Check item detection
- Verify recipe relevance

Activity 5.2: AI Output Quality

- Check ingredient accuracy
- Check recipe usefulness
- Check substitute suggestions

Activity 5.3: Performance Optimization

- Reduce image size
- Cache results
- Improve prompts

Activity 5.4: Deployment Preparation

- Create requirements.txt
- Add Dockerfile
- Test locally and in container

MILESTONE 1: AI Vision Initialization

Goal: Set up AI to detect pantry items from images.

Activity 1.1: Setup AI Platform

- Sign in to OpenAI Platform or Google AI Studio
- Create API Key
- Store key in .env file
- Select Vision model (GPT-4o mini / Gemini Vision)

Activity 1.2: Configure Environment

- Create .env file with API key

```
GEMINI_API_KEY=your_api_key_here
```

- Install dependencies

```
pip install openai streamlit pillow python-dotenv
```

- Test API connection

```
from openai import OpenAI
import os
from dotenv import load_dotenv

load_dotenv()
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role":"user","content":"Say Hello Kitchen Vision"}]
)

print(response.choices[0].message.content)
```

Activity 1.3: Test Image Detection

- Upload sample pantry image

```python
def detect_items():
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {
                "role": "user",
                "content": "List food items present in a pantry image."
            }
        ]
    )
    print(response.choices[0].message.content)

detect_items()
```

Testing image understanding

- AI returns list of detected ingredients

- rice
- pasta
- canned tomatoes
- olive oil

Well-formatted list of pantry items

MILESTONE 2: Core Functionalities

Goal: Build main AI features for Kitchen Vision.

```
vision_service.py      # Item detection
recipe_service.py      # Recipe generation
substitute_service.py  # Ingredient substitution
health_service.py      # Nutrition scoring

ls/
image_utils.py         # Image preprocessing
helpers.py             # Common utilities

ets/
sample_images/

puts/
results.json
```

File explorer structure

Activity 2.1: Streamlit App Setup

- Import Streamlit and modules
- Load environment variables
- Set page layout

Activity 2.2: Image Upload & Processing

- Upload pantry image
- Resize and enhance image
- Convert image to byte format

Activity 2.3: Pantry Item Detection

- Send image to Vision Service
- Detect ingredient names

```
["rice", "tomato", "onion", "milk", "egg", "cheese"]
```

Activity 2.4: Item Categorization

- Group items into
    - Dry
    - Cold
    - Frozen
    - Fresh

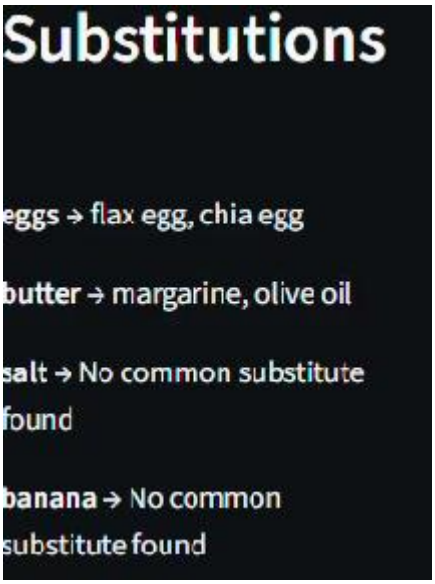
Rule based mapping using dictionary lookups.

Activity 2.5: Recipe Generation

- Generate 3 simple recipes
- Include ingredients, steps, time, difficulty

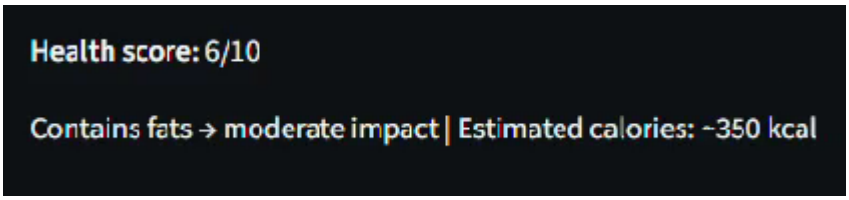
Activity 2.6: Substitution Suggestions

- Suggest alternative ingredients



Activity 2.7: Health Score

- Give health rating (1–10)
- Based on nutrition factors



Activity 2.8: Result Aggregation

Final result shows:

- Detected items
- Storage categories
- Recipes
- Substitutes
- Health score
- Grocery list

MILESTONE 3: Modular Architecture

Activity 3.1: Main Application

- Run app.py as main entry

```
if __name__ == "__main__":
    print("="*60)
    print("🍴 Kitchen Vision - Starting Server")
    print("="*60)
    print("✅ API Keys configured")
    print("✅ Using Streamlit UI")
    print("\nServer running at: http://localhost:8501")
    print("="*60)

    # Run Streamlit (frontend)
    # Use command: streamlit run app.py
```

Activity 3.2: Folder Structure

- Separate UI, services, utilities

```
KitchenVision/
│
├── app.py
├── config.py
├── services/
│   ├── vision_service.py
│   ├── recipe_service.py
│   ├── health_service.py
│   └── substitute_service.py
│
├── utils/
│   ├── image_utils.py
│   └── prompt_templates.py
├── requirements.txt
├── .env.example
└── README.md
```

Activity 3.3: Dependencies

- Create requirements.txt

```
nginx

streamlit
pillow
openai
numpy
pandas
```

.env.example

```ini
OPENAI_API_KEY=your_api_key_here
```

Activity 3.4: Setup & Run

Steps:

1. Clone project
2. Install libraries
3. Add .env API key
4. Run streamlit run app.py

```
FROM python:3.11
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
EXPOSE 8501
CMD ["streamlit", "run", "app.py", "--server.port=8501", "--server.address=0.0.0.0"]
```

Optional docker deployment

MILESTONE 4: UI Development

Activity 4.1: Landing Page

- Project title and description
- Feature cards for:
    - Pantry detection
    - Recipe generator
    - Grocery list
    - Health rating

Landing Page Features

```
import streamlit as st

st.set_page_config(page_title="Kitchen Vision", layout="wide")
st.title("🍴 Kitchen Vision")
st.subheader("Turn your pantry into recipes in seconds!")

# Feature Cards
st.markdown("""
<div style='display:flex; justify-content:space-around;'>
    <div style='text-align:center; padding:10px; border:1px solid #eee; width:23%'>
        <h4>Pantry Upload</h4>
        <p>Upload pantry images to detect items</p>
    </div>
    <div style='text-align:center; padding:10px; border:1px solid #eee; width:23%'>
        <h4>Recipe Generator</h4>
        <p>Get 3-step recipe suggestions</p>
    </div>
    <div style='text-align:center; padding:10px; border:1px solid #eee; width:23%'>
        <h4>Grocery List</h4>
        <p>Missing ingredients checklist</p>
    </div>
    <div style='text-align:center; padding:10px; border:1px solid #eee; width:23%'>
        <h4>Health Score</h4>
        <p>Nutrition rating for recipes</p>
    </div>
</div>
""", unsafe_allow_html=True)
```

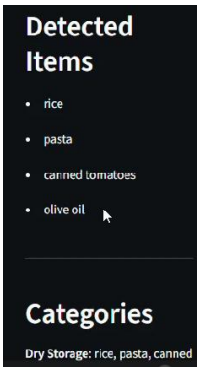
Activity 4.2: Generator Pages

Pages include:

Pantry Detection

- Upload image

- Show detected items



Recipe Generator

- Show 3 recipes

Grocery List

- Display missing ingredients

Health Rating

- Show health score and nutrition info

```
uploaded_file = st.file_uploader("Upload Pantry Image", type=["jpg","png"])
if uploaded_file:
    from utils.image_utils import preprocess_image
    image = preprocess_image(uploaded_file)
    st.image(image, caption="Uploaded Pantry", use_column_width=True)

    from services.vision_service import detect_items
    items = detect_items(image)
    st.subheader("🍅 Detected Pantry Items")
    st.json(items)
```

Activity 4.3: Navigation

Sidebar menu:

- Home
- Pantry Upload
- Recipe Generator
- Grocery List
- Health Rating

Example:

```
page = st.sidebar.selectbox("Navigate", ["Home", "Pantry Upload", "Recipe Generator", "Grocery Li

if page == "Home":
    st.write("Welcome to Kitchen Vision!")
elif page == "Pantry Upload":
    # Pantry upload code
    pass
elif page == "Recipe Generator":
    # Recipe generator code
    pass
# Continue for other pages
```

MILESTONE 5: Testing & Optimization

Activity 5.1: Functional Testing

Test:

- Pantry image detection
- Recipe generation
- Grocery list
- Health score
- UI functionality

General recipe example

Activity 5.2: Quality Evaluation

Check:

- Detection accuracy
- Recipe usefulness

- Substitute quality
- Response speed

```
1. Prepare ingredients: pasta, canned tomatoes, olive oil.

2. Cook according to basic directions for each ingredient.

3. Serve and enjoy.
```

Activity 5.3: Performance Optimization

Improvements:

- Resize images
- Cache results
- Reduce API calls
- Add loading spinner
- Error handling

```
@st.cache_data
def cached_detect_items(image):
    return detect_items(image)

with st.spinner("Analyzing Pantry..."):
    items = cached_detect_items(image)
```

Conclusion

Kitchen Vision is an AI-powered cooking assistant that analyzes pantry images, detects ingredients, generates recipes, suggests substitutes, and provides health scores.

It helps users cook smarter, reduce food waste, and plan healthier meals using AI vision and Streamlit.